CS 645

Security and Privacy in Computer Systems

Lecture 13: Security in *databases* and *outsourced databases*

- Database access control
- Protecting sensitive data
- Secure database outsourcing:
  - Database as a service
  - Authentication and integrity
  - Data confidentiality
  - Query completeness

Announcements

- Final exam: May 8, 6:00–8:30pm
  - Covers all material (lectures 1-14)
  - Closed books and closed notes

- Please complete the (online) course evaluation!
  - through Canvas: https://canvas.njit.edu/ (bottom of the menu on the left of the screen)

  or

  - by using the NJIT GO App (available from Apple App Store or Google Play)
  - Takes less than 3 minutes to complete

- Project #3 is due next week, on Apr 24

Last week on CS 645

- Rootkits
- Rootkits case study: XCP
- Zero-day attacks
- Patch distribution schemes
- Botnets
- Recent cybersecurity incidents

Database access control

Database Access Control

- A proper set of database access controls should implement a **least-privilege principle**, so that each user has the necessary rights to perform their required tasks, but no rights beyond that.
- A proper set of database access controls should also implement a **separation of privilege principle**, so that different users have different privileges, depending on the different tasks that they need to perform.

Access Control Using SQL

- SQL defines an access control framework that is commonly used for defining database privileges.
- When a table is created, the owner of the table has the sole rights to perform operations on that table.
- The owner can then **grant privileges** to other users, which is known as **privilege delegation.**
- These privileges may be broad, such as the ability to do anything to a particular table, or fine-grained, such as the ability to perform only SELECT queries on certain columns.

Access Control Using SQL

- For example, the owner of a table may issue the following SQL command to give Alice the ability to search through table employees:

```
GRANT SELECT ON employees TO Alice;
```

- Other permissions that can be provided using the GRANT keyword include DELETE, INSERT, and UPDATE. In addition, to grant all available rights one can use the ALL keyword.

Privilege Delegation

- **Policy authority delegation:** table owners can also allow other users to grant privileges for those tables
  - The grantor can include the clause WITH GRANT OPTION to give the recipient the ability to further delegate that privilege.
  - For example, an administrator might create a view for Alice and give her permission to delegate SELECT permissions on that view to other users as follows:

```
CREATE VIEW employees_alice AS
SELECT * FROM employees
WHERE name = `Alice';
GRANT SELECT ON employees_alice TO Alice WITH
GRANT OPTION;
```

Privilege Revocation

- The propagation of privileges in a database can be visualized using a diagram, where nodes represent users and directed edges represent granted privileges.
- If Alice grants a set of rights, S, to Bob, then we draw a directed edge labeled with S from Alice to Bob.
- A user, Alice, who has granted privileges to another, Bob, can opt to revoke those privileges at a later time, which would be visualized by deleting or relabeling the edge from Alice to Bob.

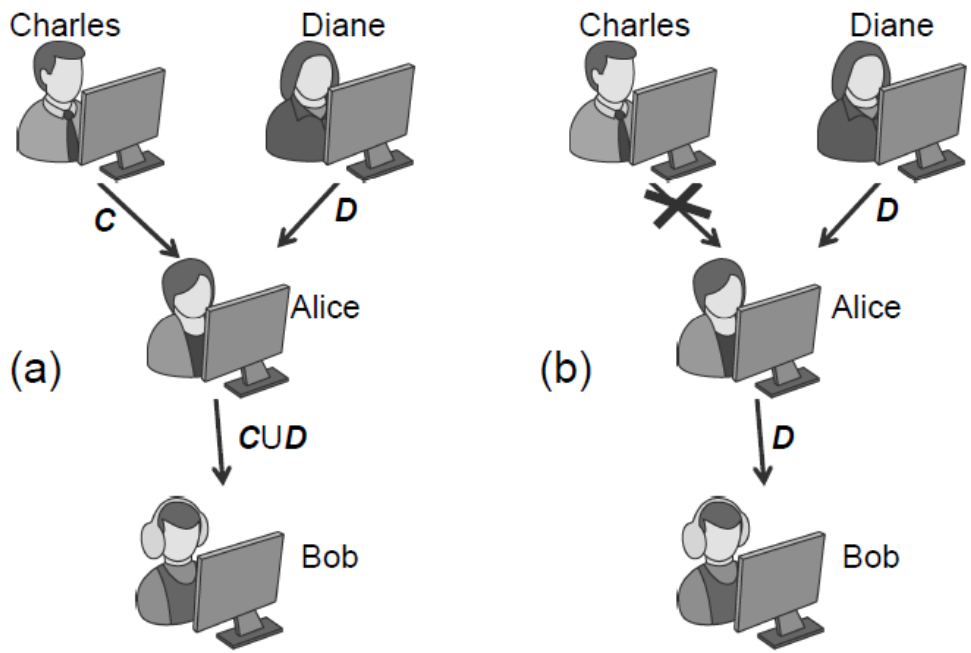
Privilege Revocation

- A command that could perform such a revocation is as follows:

  REVOKE SELECT ON employees FROM Bob;

- This command should result in the revocation of all SELECT privileges for Bob as well as all the people to which he had delegated this privilege.

Privilege Revocation Example



(a) First, two administrators, Charles and Diane, each grant Alice two sets of privileges, C and D, after which Alice grants those privileges to Bob, giving him the set of rights in the union, C U D.

(b) If Charles subsequently revokes the set of privileges, C, he granted to Alice, then the privileges Bob inherited indirectly from Charles, through Alice, should also be revoked, leaving Bob with just the privileges in D.

Propagating privilege revocation

- How to implement privilege revocation propagation?
  - The principle is that **the privilege given to users should be the same as if the revoked privilege had never been granted**

- Can be efficiently implemented by using a timestamp

Grantor	Grantee	Privilege	Timestamp
Alice	Carol	P	1
Bob	Carol	P	2
Carol	David	P	3

- At time 5, Alice revokes P from Carol. Does Carol still has privilege P? What happens to David?

Propagating privilege revocation

- Imagine that Bob gave privilege P to Carol at time 4 (instead of 2). What happens to David when Alice revokes P from Carol at time 5? What about Carol?

Grantor	Grantee	Privilege	Timestamp
Alice	Carol	P	1
Carol	David	P	3
Bob	Carol	P	4

Protecting sensitive data

Sensitive Data

- In addition to ensuring that databases have appropriate access-control measures in place, care must be taken to guarantee that sensitive data is stored in a way that protects the privacy of users and any confidentiality requirements for sensitive data.

Using Cryptography

- If information being stored in a database has confidentiality requirements, then it should not be stored in plaintext, but should instead be stored as the output of a cryptographic function.
- Confidential information kept in a database should be stored in encrypted form, where the decryption key should be known by authorized users but not stored in the database itself.

Privacy Protection

- Database owners should also be careful to consider the privacy impacts of *publishing* or *granting access* to sensitive information.
- If a database is to be *released to the public*, say, to be used for research purposes, then
  - all identifying information, such as names, addresses, Social Security numbers, employee numbers, and student numbers, should be removed or changed to **masking values,** which are nondescript values that lack all identifying information.

Inference Attacks

- **Inference attacks**: Even if identifying information is removed or masked out, it may still be possible to use the database in conjunction with additional information available to the attacker to learn more about the underlying data.
- Example: consider a database of employee records, whose attributes are name, gender, ID number, and salary.
  - Suppose a party is granted access to a sanitized version of the table, where the name attribute is removed, for the purpose of creating statistics on salary by gender. Another party may have a list of pairings associating ID numbers to names for a reporting task.
  - If these two parties were to communicate, they could easily infer the salary of each employee, despite the intent of the database owner. In general, when granting access to modified versions of a database, administrators should consider whether ***collusion among grantees*** can allow them to gain unauthorized information.

Protecting Against Inference Attacks

- To protect a database from inference attacks, the following techniques can be used prior to making the database public.
  - **Cell suppression.** In using this technique, some of the cells in a database are removed and left blank in the published version.
  - **Generalization.** In using this technique, some values in a published database are replaced with more general values.
  - **Noise addition.** In using this technique, values in a published database have random values added to them, so that the noise across all records for the same attribute averages out to zero.

Example

Num	Age1	Age2
11	49.3	53.6
18	46.9	63.2
20	49.3	49.8
35	43.6	46.5
42	46.4	
44	47.5	

(a)

Num	Age1	Age2
11	49.3	
18	46.9	63.2
20	49.3	
35		
42	46.4	
44	47.5	

(b)

Num	Age1	Age2
11	45–50	50–60
18	45–50	60–75
20	45–50	45–50
35	40–45	45–50
42	45–50	
44	45–50	

(c)

Num	Age1	Age2
11	47.7	55.2
18	49.2	64.3
20	51.6	52.8
35	42.3	47.3
42	47.1	
44	48.0	

(d)

Figure 10.4: Obfuscation techniques for protecting the privacy of individuals included in a public database: (a) A table with individual names removed. (b) A table anonymized using cell suppression. (c) A table anonymized using generalization. (d) A table anonymized using noise.

How much data to suppress?

- One extreme: “blur” the data so much that it becomes unusable to compute any statistics
- Other extreme: not suppress enough data, and sensitive information can be inferred
- Unfortunately, there is no widely accepted standard for deciding when information in a public database has been sufficiently obscured (sanitized)
  - k-anonymization: a database is considered sufficiently anonymized if any possible SELECT query returns at least k records, where k is a large enough threshold of disclosure tolerance

k-anonymization

Motivation

- Main Question:
  - How can a data holder release stored information without compromising privacy?
- Challenge:
  - On one hand, the released data must be useful and on the other hand we must provide proper protection to the data.

Motivation

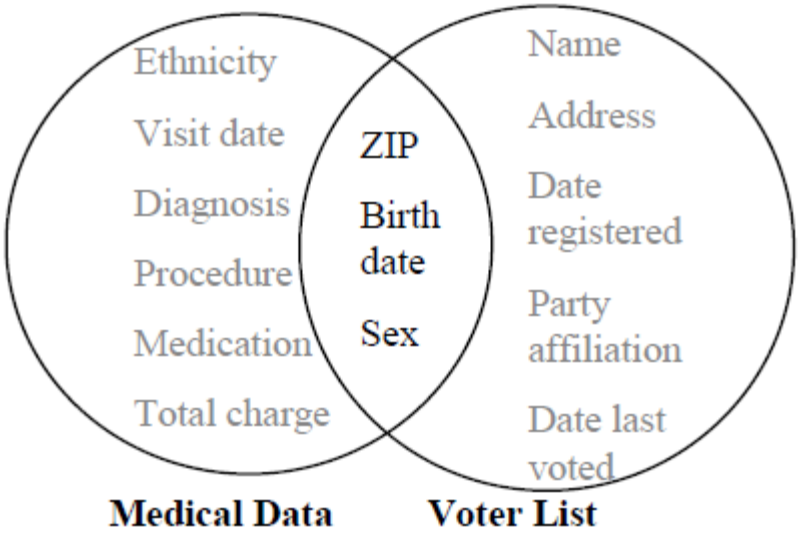
- Common practice:
  - Release person specific data after removing all explicit identifiers like name, address and phone number assuming data is anonymous.

- Problem is:
  - Individuals related to data can be re-identified by linking or matching with other publicly available data.

Motivation

- **For example:** 87% of US population can be uniquely identified based on **{5-digit ZIP, gender, date of birth}**

- Re-identification by linking
  - National Association of Health Data Organizations (NAHDO) – collects hospital level data
    (Left-most circle)
  - Voter registration list
    (Right-most circle)



Motivation

- William Weld was governor of Massachusetts, whose medical records were also in the same medical data.
  - He lived in Cambridge Massachusetts.
  - According to the Cambridge Voter list,
    - six people had his particular birth date;
    - only three of them were men; and,
    - he was the only one in his 5-digit ZIP code

Question

- How can a data holder release a version of its private data?
  - with scientific guarantees that the individuals who are the subjects of the data cannot be re-identified.
  - while the data remain practically useful.

Background

- Statistical databases
  - Aggregated data stored for applications like
    - Data mining
    - Cost analysis
    - Fraud detection
    - Research
- For anonymity
  - Adding noise to data
- Problem
  - Destroy the integrity of records, so for many new uses of data, these techniques are not appropriate

Background

- Restricting release of lower classified information so that higher classified information cannot be derived.
  - **Multi-level databases**
    - data stored at different security classifications
    - users having different security clearances

- Information is divided and partially replicated among multiple holders
  - **Problem:** Data holders make decisions locally with incomplete knowledge on others data and release data individually.

- Primary Solution: **Suppression**
    - Sensitive information is simply not released
    - This loss of data makes data useless

Background

- Security
  - Access control & Authentication
    - Safeguard direct disclosures
    - But cannot control disclosures based on inferences

- Multiple queries can leak inference
  - Table (physician, patient, medication)
    - query listing the patients seen by each physician
    - query itemizing medications prescribed by each physician

  - Problem:
    - query associating patients with their prescribed medications may be sensitive because medications typically correlate with diseases

Goal

- To provide a model for
  - understanding,
  - evaluating and
  - constructing computational systems that control inferences.

Definitions

- Table consists of
  - Attributes
  - Tuples

Attribute names ------►
Row (tuple) ------►

cFirstName	cLastName	cPhone	cStreet	cZipCode
Tom	Jewett	714-555-1212	10200 Slater	92708

- Let $B(A_1,\ldots,A_n)$ be a table with a finite number of tuples.
  - The finite set of **attributes** of B are $\{A_1,\ldots,A_n\}$.
  - $t[A_i,\ldots,A_j]$ – single tuple
  - $B[A_i,\ldots,A_j]$ – projection

Definitions

- Inference
  - To believe a new fact on the basis of other information
- Disclosure
  - When explicit or inferable information about a person is released that was not intended.
- Disclosure control
  - to identify and limit disclosures in released data

Definitions

- **Quasi-identifier:**
  - **All attributes in the private information that could be used for linking with external information.**

- Let V be the voter-specific table as the voter list.
  - A quasi-identifier for V, written Q_V, is
    - **{name, address, ZIP, birth date, gender}**

Assumption

- **Assumption: Quasi-identifier can always be identified accurately**
  - The data holder can identify attributes in his private data that may also appear in external information and therefore, can accurately identify quasi-identifiers

k-anonymity protection model

- $RT(A_1,...,A_n)$ is a table
- QI_{RT} is the quasi-identifier
- **RT is said to satisfy k-anonymity**
  - if and only if each sequence of values in $RT[QI_{RT}]$ appears with at least k occurrences in $RT[QI_{RT}]$.

k-anonymity

- Example of k-anonymity, for
  - k=2
  - QI={Race, Birth, Gender, ZIP}

	Race	Birth	Gender	ZIP	Problem
t1	Black	1965	m	0214*	short breath
t2	Black	1965	m	0214*	chest pain
t3	Black	1965	f	0213*	hypertension
t4	Black	1965	f	0213*	hypertension
t5	Black	1964	f	0213*	obesity
t6	Black	1964	f	0213*	chest pain
t7	White	1964	m	0213*	chest pain
t8	White	1964	m	0213*	obesity
t9	White	1964	m	0213*	short breath
t10	White	1967	m	0213*	chest pain
t11	White	1967	m	0213*	chest pain

k-anonymity

- There are at least 2 tuples that have the same values for the set of quasi-identifiers
  - $t1[QI_T] = t2[QI_T]$,
  - $t3[QI_T] = t4[QI_T]$,
  - $t5[QI_T] = t6[QI_T]$,
  - $t7[QI_T] = t8[QI_T] = t9[QI_T]$,
  - $t10[QI_T] = t11[QI_T]$

	Race	Birth	Gender	ZIP	Problem
t1	Black	1965	m	0214*	short breath
t2	Black	1965	m	0214*	chest pain
t3	Black	1965	f	0213*	hypertension
t4	Black	1965	f	0213*	hypertension
t5	Black	1964	f	0213*	obesity
t6	Black	1964	f	0213*	chest pain
t7	White	1964	m	0213*	chest pain
t8	White	1964	m	0213*	obesity
t9	White	1964	m	0213*	short breath
t10	White	1967	m	0213*	chest pain
t11	White	1967	m	0213*	chest pain

k-anonymity

- k occurrences of each value under k-anonymity
  - |T[Race="black"]| = 6
  - |T[Race="white"]| = 5
  - |T[Birth="1964"]| = 5
  - |T[Birth="1965"]| = 4
  - |T[Birth="1967"]| = 2
  - |T[Gender="m"]| = 6
  - |T[Gender="f"]|= 5
  - |T[ZIP="0213*"]| = 9
  - |T[ZIP="0214*"]| = 2

	Race	Birth	Gender	ZIP	Problem
t1	Black	1965	m	0214*	short breath
t2	Black	1965	m	0214*	chest pain
t3	Black	1965	f	0213*	hypertension
t4	Black	1965	f	0213*	hypertension
t5	Black	1964	f	0213*	obesity
t6	Black	1964	f	0213*	chest pain
t7	White	1964	m	0213*	chest pain
t8	White	1964	m	0213*	obesity
t9	White	1964	m	0213*	short breath
t10	White	1967	m	0213*	chest pain
t11	White	1967	m	0213*	chest pain

k-anonymity

- Property:
  - Combining RT table and external sources on QI_{PT} or a subset of its attributes, the released data will match to k individuals or more.
  - This property protects table *against inference from linking* (by direct matching) to known external sources
- Limitation:
  - This property does not guarantee that “individuals cannot be identified in RT” (there may be many other inference attacks!)
- Protects only against *inference from linking* attack.

Attacks

- 3 attacks against k-anonymity
  - Unsorted matching attack
  - Complementary release attack
  - Temporal attack

Unsorted matching attack

- Attack is based on the order in which tuples appear in the released table.
- Tables GT1 and GT2 are based on PT and adhere to k-anonymity, where
  - QI_{PT} = {Race, ZIP} and k=2.

Race	ZIP
Asian	02138
Asian	02139
Asian	02141
Asian	02142
Black	02138
Black	02139
Black	02141
Black	02142
White	02138
White	02139
White	02141
White	02142

PT

Race	ZIP
Person	02138
Person	02139
Person	02141
Person	02142
Person	02138
Person	02139
Person	02141
Person	02142
Person	02138
Person	02139
Person	02141
Person	02142

GT1

Race	ZIP
Asian	02130
Asian	02130
Asian	02140
Asian	02140
Black	02130
Black	02130
Black	02140
Black	02140
White	02130
White	02130
White	02140
White	02140

GT2

Unsorted matching attack

- Direct matching of tuples across the tables based on tuple position reveals sensitive information.

Race	ZIP
Asian	02138
Asian	02139
Asian	02141
Asian	02142
Black	02138
Black	02139
Black	02141
Black	02142
White	02138
White	02139
White	02141
White	02142

PT

Race	ZIP
Person	02138
Person	02139
Person	02141
Person	02142
Person	02138
Person	02139
Person	02141
Person	02142
Person	02138
Person	02139
Person	02141
Person	02142

GT1

Race	ZIP
Asian	02130
Asian	02130
Asian	02140
Asian	02140
Black	02130
Black	02130
Black	02140
Black	02140
White	02130
White	02130
White	02140
White	02140

GT2

- Can be corrected by randomly sorting the tuples of the released table GT1 and GT2

Complementary release attack

- Using subsequent releases of the same privately held information, attacker can relate the data and can infer new information.

- Private Table PT

Race	BirthDate	Gender	ZIP	Problem
black	9/20/1965	male	02141	short of breath
black	2/14/1965	male	02141	chest pain
black	10/23/1965	female	02138	painful eye
black	8/24/1965	female	02138	wheezing
black	11/7/1964	female	02138	obesity
black	12/1/1964	female	02138	chest pain
white	10/23/1964	male	02138	short of breath
white	3/15/1965	female	02139	hypertension
white	8/13/1964	male	02139	obesity
white	5/5/1964	male	02139	fever
white	2/13/1967	male	02138	vomiting
white	3/21/1967	male	02138	back pain

PT

Complementary release attack

- GT1 is released based on PT and adhere to k-anonymity, where
  - k=2 and QI_{PT}= {Race, BirthDate, Gender, ZIP}.
- Subsequently GT3 is also released (GT3 respects the same k-anonymity)

Race	BirthDate	Gender	ZIP	Problem
black	9/20/1965	male	02141	short of breath
black	2/14/1965	male	02141	chest pain
black	10/23/1965	female	02138	painful eye
black	8/24/1965	female	02138	wheezing
black	11/7/1964	female	02138	obesity
black	12/1/1964	female	02138	chest pain
white	10/23/1964	male	02138	short of breath
white	3/15/1965	female	02139	hypertension
white	8/13/1964	male	02139	obesity
white	5/5/1964	male	02139	fever
white	2/13/1967	male	02138	vomiting
white	3/21/1967	male	02138	back pain

PT

Race	BirthDate	Gender	ZIP	Problem
black	1965	male	02141	short of breath
black	1965	male	02141	chest pain
person	1965	female	0213*	painful eye
person	1965	female	0213*	wheezing
black	1964	female	02138	obesity
black	1964	female	02138	chest pain
white	1964	male	0213*	short of breath
person	1965	female	0213*	hypertension
white	1964	male	0213*	obesity
white	1964	male	0213*	fever
white	1967	male	02138	vomiting
white	1967	male	02138	back pain

GT1

Race	BirthDate	Gender	ZIP	Problem
black	1965	male	02141	short of breath
black	1965	male	02141	chest pain
black	1965	female	02138	painful eye
black	1965	female	02138	wheezing
black	1964	female	02138	obesity
black	1964	female	02138	chest pain
white	1960-69	male	02138	short of breath
white	1960-69	human	02139	hypertension
white	1960-69	human	02139	obesity
white	1960-69	human	02139	fever
white	1960-69	male	02138	vomiting
white	1960-69	male	02138	back pain

GT3

Complementary release attack

- Linking GT1 and GT3 on {Problem} reveals the table LT
- Unique tuples in LT are inferred: [white,1964,male,02138] & [white,1965,female,02139]

Race	BirthDate	Gender	ZIP	Problem
black	9/20/1965	male	02141	short of breath
black	2/14/1965	male	02141	chest pain
black	10/23/1965	female	02138	painful eye
black	8/24/1965	female	02138	wheezing
black	11/7/1964	female	02138	obesity
black	12/1/1964	female	02138	chest pain
white	10/23/1964	male	02138	short of breath
white	3/15/1965	female	02139	hypertension
white	8/13/1964	male	02139	obesity
white	5/5/1964	male	02139	fever
white	2/13/1967	male	02138	vomiting
white	3/21/1967	male	02138	back pain

PT

Race	BirthDate	Gender	ZIP	Problem
black	1965	male	02141	short of breath
black	1965	male	02141	chest pain
black	1965	female	02138	painful eye
black	1965	female	02138	wheezing
black	1964	female	02138	obesity
black	1964	female	02138	chest pain
white	1964	male	02138	short of breath
white	1965	female	02139	hypertension
white	1964	male	02139	obesity
white	1964	male	02139	fever
white	1967	male	02138	vomiting
white	1967	male	02138	back pain

LT

Race	BirthDate	Gender	ZIP	Problem
black	1965	male	02141	short of breath
black	1965	male	02141	chest pain
person	1965	female	0213*	painful eye
person	1965	female	0213*	wheezing
black	1964	female	02138	obesity
black	1964	female	02138	chest pain
white	1964	male	0213*	short of breath
person	1965	female	0213*	hypertension
white	1964	male	0213*	obesity
white	1964	male	0213*	fever
white	1967	male	02138	vomiting
white	1967	male	02138	back pain

GT1

Race	BirthDate	Gender	ZIP	Problem
black	1965	male	02141	short of breath
black	1965	male	02141	chest pain
black	1965	female	02138	painful eye
black	1965	female	02138	wheezing
black	1964	female	02138	obesity
black	1964	female	02138	chest pain
white	1960-69	male	02138	short of breath
white	1960-69	human	02139	hypertension
white	1960-69	human	02139	obesity
white	1960-69	human	02139	fever
white	1960-69	male	02138	vomiting
white	1960-69	male	02138	back pain

GT3

Complementary release attack

- This problem would not exist
  - if GT3 used the quasi-identifier {QI} U {*Problem}*

  or
  - if GT1 had been the basis of GT3

Main problem?

- How can you know in advance which are the quasi-identifiers?
  - k-anonymity assumes the QIs are easy to identify
  - But tables are dynamic, and external information is dynamic, so correctly identifying QIs is challenging

- Full article:

https://epic.org/privacy/reidentification/Sweeney_Article.pdf

Secure database outsourcing: Database as a service

Database as a Service

- Outsource the storage and management of the DB to a third party
- Third party offers all DB functions transparently to the client

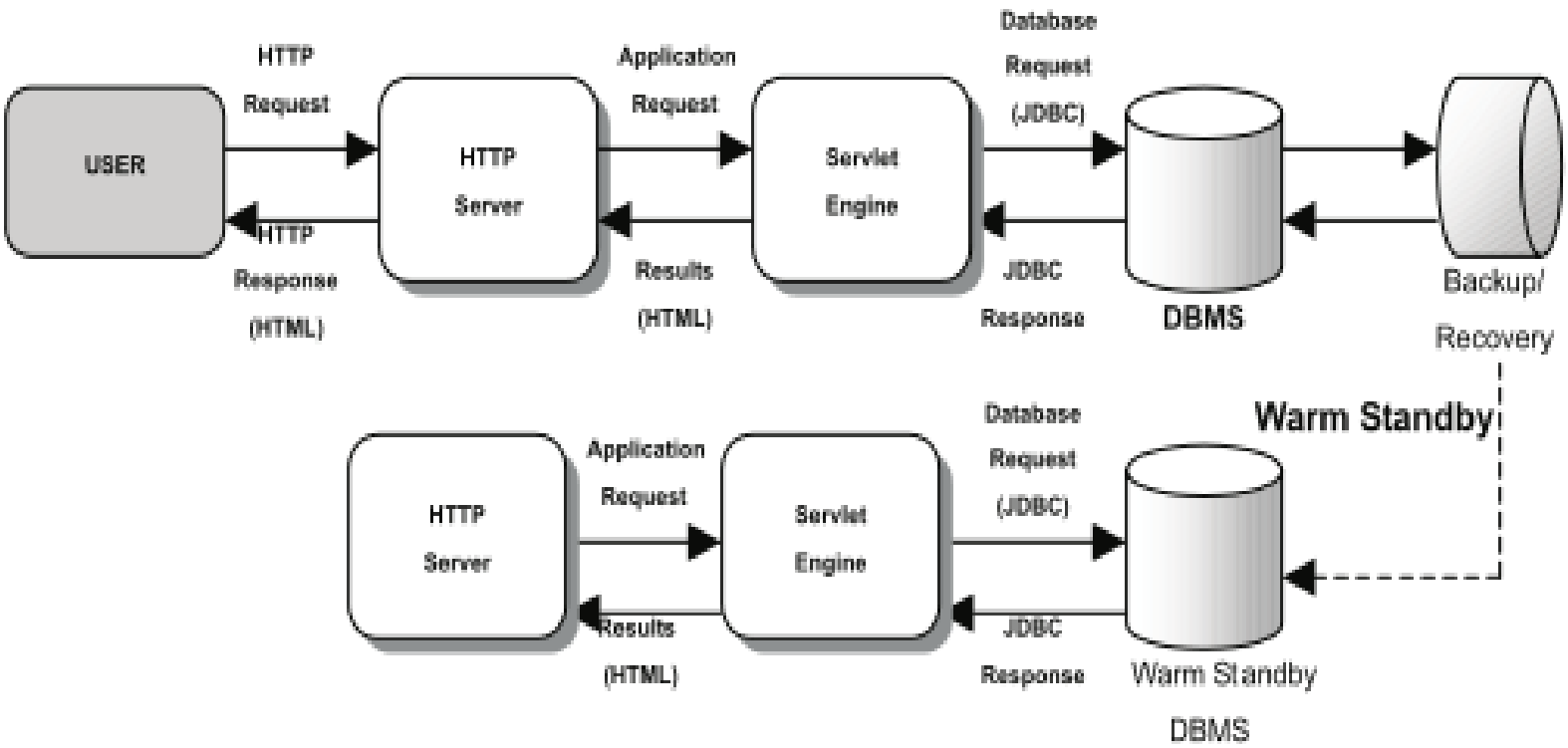
Motivation for database as a service

- Initial set up cost
  - Installing hardware and software
  - Establishing network connectivity
  - Deploying database products
- Database management cost is high
  - Database back up
  - Hiring database administrators
  - Database restoration
  - Database reorganization to reclaim space
  - Migration from one database version to next
- Conclusion: it's cheaper to outsource than to host
  - Reduction in hardware, software, personnel costs
  - Economies of scale

New DB Management Model

- Hacigumus et al., "*Providing Database as a Service*", in IEEE ICDE 2002
- Previous mentioned tasks are provided by a Service Provider
- Applications are not impacted by software, hardware and network changes at database service provider site
- Organization gets ready-to-use system, maintained by the service provider

NetDb2 System Architecture



Data Privacy

- Privacy of data during **network** transmission
  - Secure Socket Layer (SSL)
  - Transport Layer Security protocol (TLS)

- Privacy of **stored data**
  - Based on encryption techniques

Encryption at Field Level

- It is used to encrypt data in one or more fields
- When data is inserted into chosen field, value is encrypted before being stored
- On read access, stored data is decrypted before being operated
- User provides the secret key, and server provides the encryption/decryption algorithm

Encryption at Field Level

- Consider the following SQL command for column discount of table lineitem
- To **insert** data into table lineitem:

```
INSERT into lineitem(discount)
VALUES (encrypt (10,key))
```

- To **select** the encrypted field:

```
SELECT decrypt (discount,key)
FROM lineitem
WHERE custid=300
```

Encryption at Field Level

- Here encrypt and decrypt functions are used
- **Creator** of encrypted data provides the **key**
- **Only users** who have **keys** can decrypt the data using decryption algorithm
- Service provider is trusted to provide the correct encryption and decryption services, and not to reveal the key
- Notice that the server has access to the secret key only for a brief interval, while performing the encryption/decryption operation
  - Thus, even is server storage is compromised, attacker will not get access to secret key

Secure database outsourcing: Authentication and integrity

Outsourced DB (ODB) Model

- Outsourced Database (ODB)
  - Service Provider offers seamless mechanisms to create, store, update and access DB
  - Service Provider is potentially untrusted
- Securing the Data
  - Providing efficient mechanisms to ensure data integrity and authenticity
  - Keeping computation and bandwidth overhead at a minimum

Data Security Issues

- Mykletun et al., "*Authentication and Integrity in Outsourced Databases*", in ISOC NDSS 2004
- Data Privacy
  - Important, but not the focus of this article
- Data Authentication
- Data Integrity
- Completeness: correct execution of the query over the entire target domain
  - Important, but not the focus of this article

Adversary Model

- Outsider attacks
  - Protect data in transit between client and server
  - Usually done with SSL/TLS
- Insider attacks
  - Service provider is untrusted with the authenticity and integrity of the data

Completeness

- Correct execution of the query over the entire target domain
  - Are all records satisfying a query returned?
- Security concern?
  - Beyond the scope of this article

Type of Functionality

- Provide standard equality SQL query
  - SELECT clause – selection of rows or attributes
- Do not consider aggregation SQL queries
  - AVERAGE OR SUM clause

Client-Server System Model

- Client
  - Administrative entity
  - An organization
  - Set of authorized users within an organization
- Server
  - Hosts outsourced database
  - Provides efficient mechanisms for clients to create, store, update and query the database
  - Relied upon for the replication, backup and availability of ODB

Client-Server System Model….

- Thin (lightweight) client
  - Computationally weak
  - Storage challenged
  - Limited Bandwidth
  - Limited Power (battery-powered)

  - Example: cellular phone, PDA etc.

Client-Server Relationship

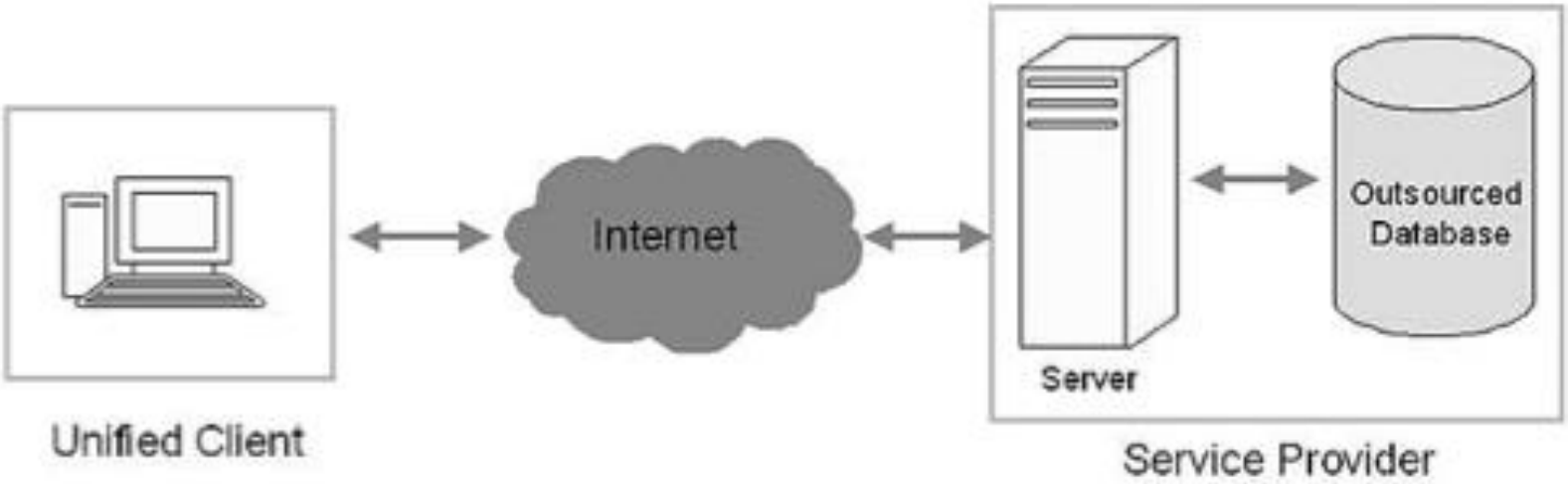
- Client trusts the server to faithfully maintain its data
  - Replication
  - Back up
  - Availability of ODB
- Server not trusted with the authenticity & integrity of the data

ODB Models

- Unified Client Model (UCM)
- Multi-Querier Model (MCM)
- Multi-Owner Model (MOM)

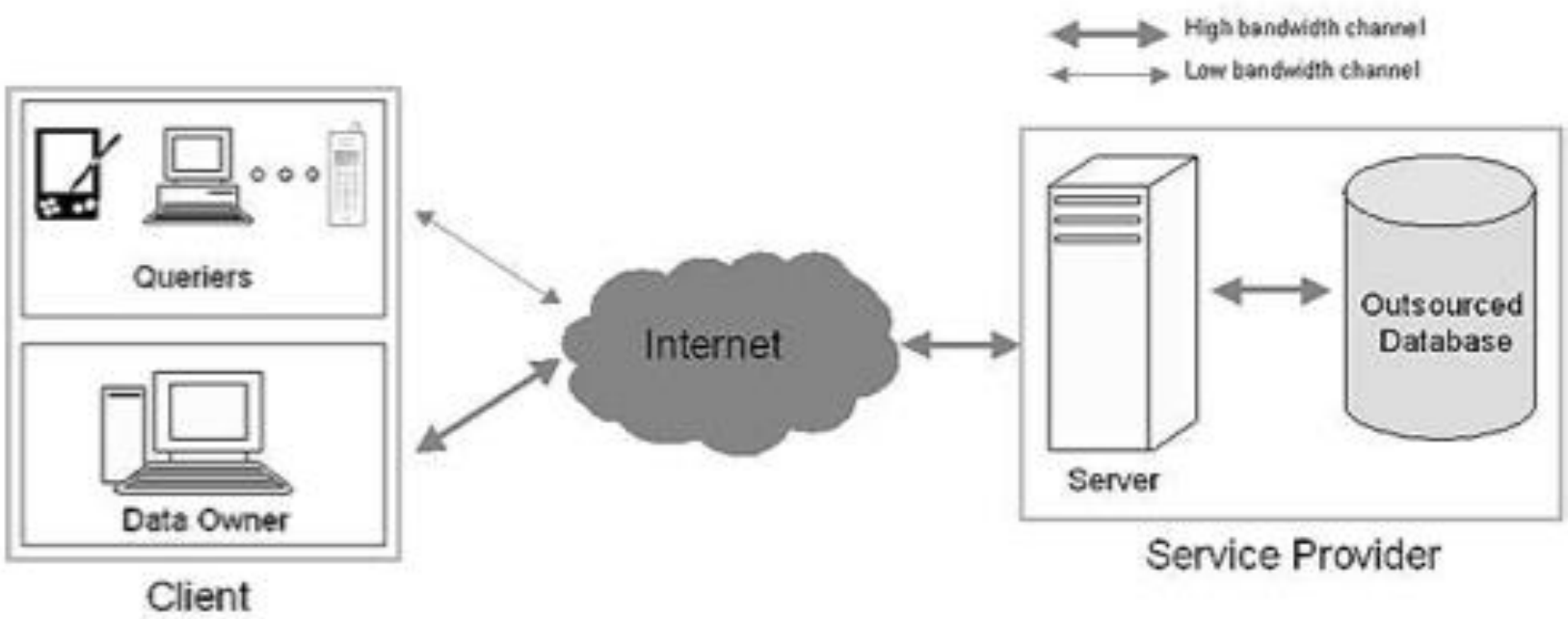
Unified Client Model

- Unified Client Model (UCM) – ODB is used by a single entity, the client who creates, manipulates and queries the data.



Multi-Querier Model

- Multi-Querier Model (MQM) - There are two types of clients, the owner and the queriers.
    - The owner adds, deletes and updates database records.
    - The querier is a client only allowed read access to the database

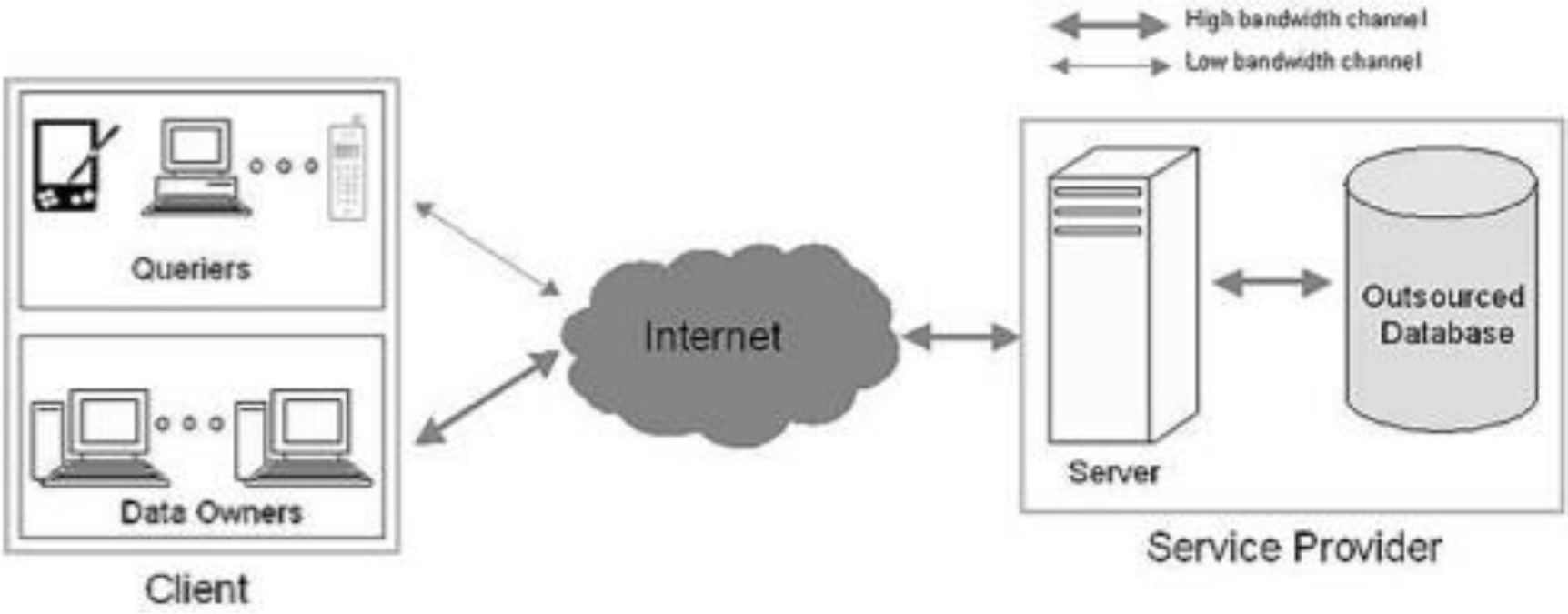


Multi-Querier Model

- Multi-Querier Model Example
  - Airlines customers checking for the availability of flight

Multi-Owner Model

- Multi-Owner Model (MOM) – a single database can have multiple owners (as well as multiple queriers)



Multi-Owner Model

- Multi-Owner Model Example
  - Customer/sales database – Each salesperson is responsible for the records that he/she creates (all the records are in the same database)

Why care about integrity/authenticity over privacy?

- Data Authentication/Integrity vs. Privacy
  - Data privacy in ODB model is orthogonal to data authentication and integrity
  - Sometimes, Authentication/Integrity takes precedence over privacy
  - Example: US Patent and Trademarks Office outsources the Patent DB which contains public data

Main reason: Public vs. Private Databases

Goals

- THE MAIN PROBLEM
  - How to facilitate secure and efficient authentication of all possible query replies when a client queries its outsourced data?
- THE GOAL
  - Assure ODB clients that data they receive from the server has not been tampered with by either an external adversary or by the server itself.

Message Authentication Codes

- MACs
  - Small & efficient to compute and verify.
- At the client end
  - Client asks server to store
    - MAC along with each record
    - Key known only to the client/queriers
- At the server end
  - For any query reply
    - The server encloses an integrity check computed over all the MACs of the records in the query reply

MAC-based solution

Setup Phase:

Owner computes

R_1 $T_1 = MAC_K(R_1)$

⋮

R_n $T_n = MAC_K(R_n)$

Owner → Server: DB & MACs

MAC-based solution

Query Phase:

- Querier -> Server: SQL Query
- Server evaluates query and obtains m records: $R_{i1}, R_{i2}, \ldots, R_{im}$
- Server computes $P = h(T_{i1} \,||\, T_{i2} \,||\, \ldots \,||\, T_{im})$
  - h is a cryptographic hash function
- Server returns to querier $(R_{i1}, R_{i2}, \ldots, R_{im}, P)$

- Querier verifies if
  $h(MAC_K(R_{i1}) \,||\, MAC_K(R_{i2}) \,||\, \ldots \,||\, MAC_K(R_{im})) \stackrel{?}{=} P$

Digital signature-based solution

Setup Phase:

Owner computes

R_1 $\sigma_1 = Sig_{Priv}(R_1)$

⋮

R_n $\sigma_n = Sig_{Priv}(R_n)$

Owner → Server: DB & signatures

Digital signature-based solution

Query Phase:

- Querier -> Server: SQL Query
- Server evaluates query and obtains m records: $R_{i1}, R_{i2}, \ldots, R_{im}$
- Server returns to querier

  $(R_{i1}, \sigma_{i1}, R_{i2}, \sigma_{i2}, \ldots, R_{im}, \sigma_{im})$

- Querier verifies if :

  σ_{i1} is a valid signature for R_{i1} ?

  …

  σ_{im} is a valid signature for R_{im} ?

Optimized solutions

- For more optimized solutions, including “Batch RSA” and “Condensed RSA”, see article.

Secure database outsourcing: Data confidentiality

Problem Description

- Hacigumus et al., "*Executing SQL over Encrypted Data in the Database Service Provider Model*", ACM SIGMOD 2002
- Privacy challenges:
  - Service providers need to protect themselves from theft of customer data
  - Total data privacy – Protection from the service providers (service provider is untrusted)
- Given a database, ensure:
  - Protection (data confidentiality) from the service provider

Challenges

- Trivial way - Transmit entire encrypted table to the client, client decrypts it
  - Negates most benefits of using a service provider
- Try to do as much of the query as possible at the provider
- All decryption still done at the client

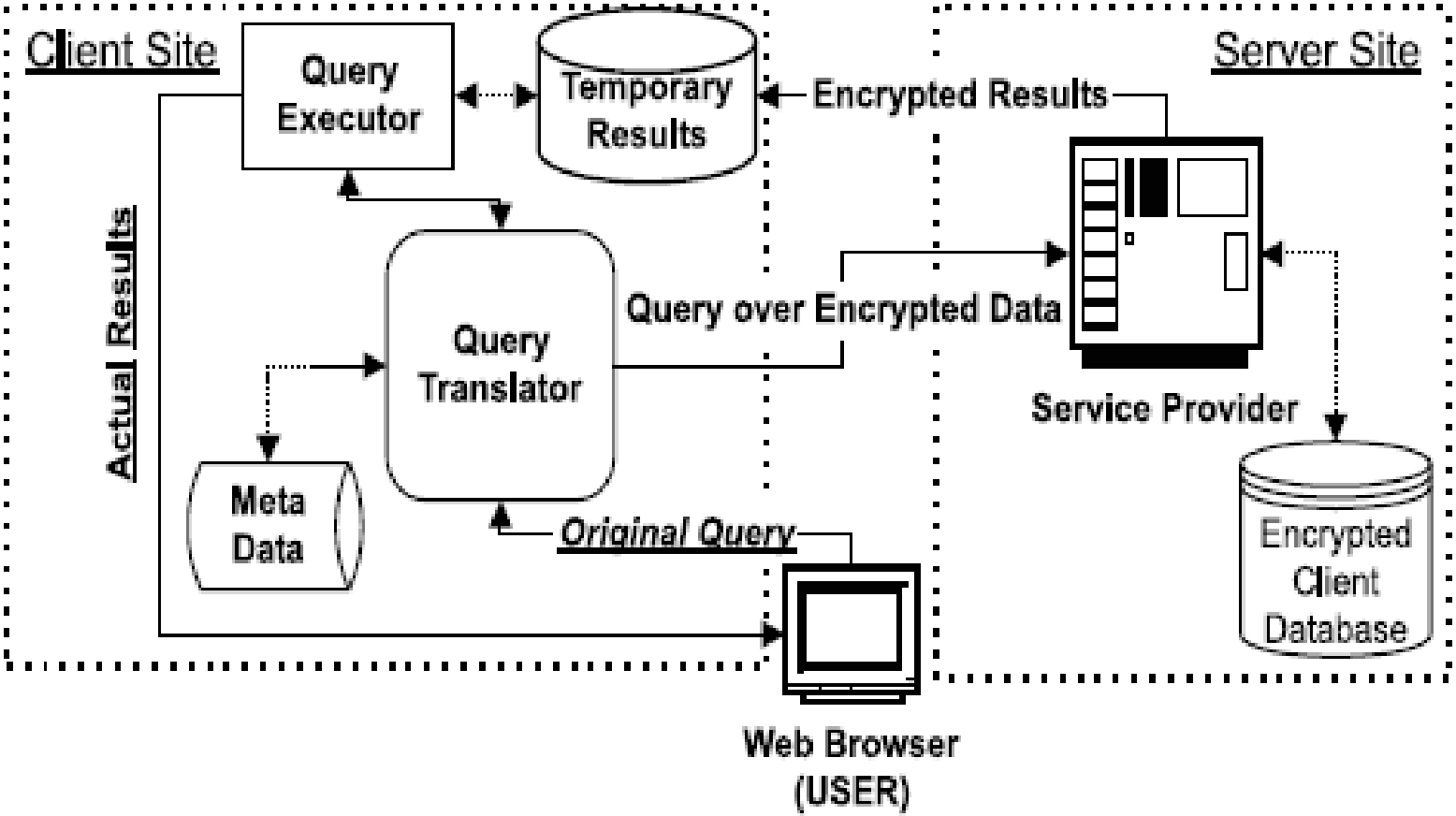
Salient Features of the Paper

- Database is stored on a server with a service provider
- Service provider has no idea about the data, since its encrypted
- Server computes part of the query
- Returns a superset of the actual answers
- Client simply filters out the returned results to get actual results
- Total privacy is achieved at the same time upholding the advantages of database as a service

General Idea

- If the database is stored encrypted at the server, server has no idea about the contents
- However, if certain metadata is assigned to each tuple, server can make a guess at the results
- Have to ensure that the metadata allows the server to narrow down results as much as possible

Basic Architecture



Basic Architecture

- Service provider has server that hosts the encrypted database
- Database is augmented with additional information
- User poses query to the client
- Client translates user query using some metadata
- Certain amount of querying done at the server
- Client post-processes query results
- Client returns the final results to the user

Basic Architecture

- Original query over unencrypted data is split into two:
  - Corresponding query over encrypted data (run on the server)
  - Client query for post-processing results from the server

- Gory details about query processing are in the article
  - Main idea is that client computes and stores additional metadata on the server
  - Server uses this metadata to answer client's queries
  - Metadata should not reveal information about the underlying corresponding data

Implemented Functionality

- Article offers solutions for secure:
  - Selections
    - Equality, range operators
  - Joins
  - Projection
  - Sorting
  - Grouping and aggregation
  - Set difference
  - Union

Secure database outsourcing: Query completeness

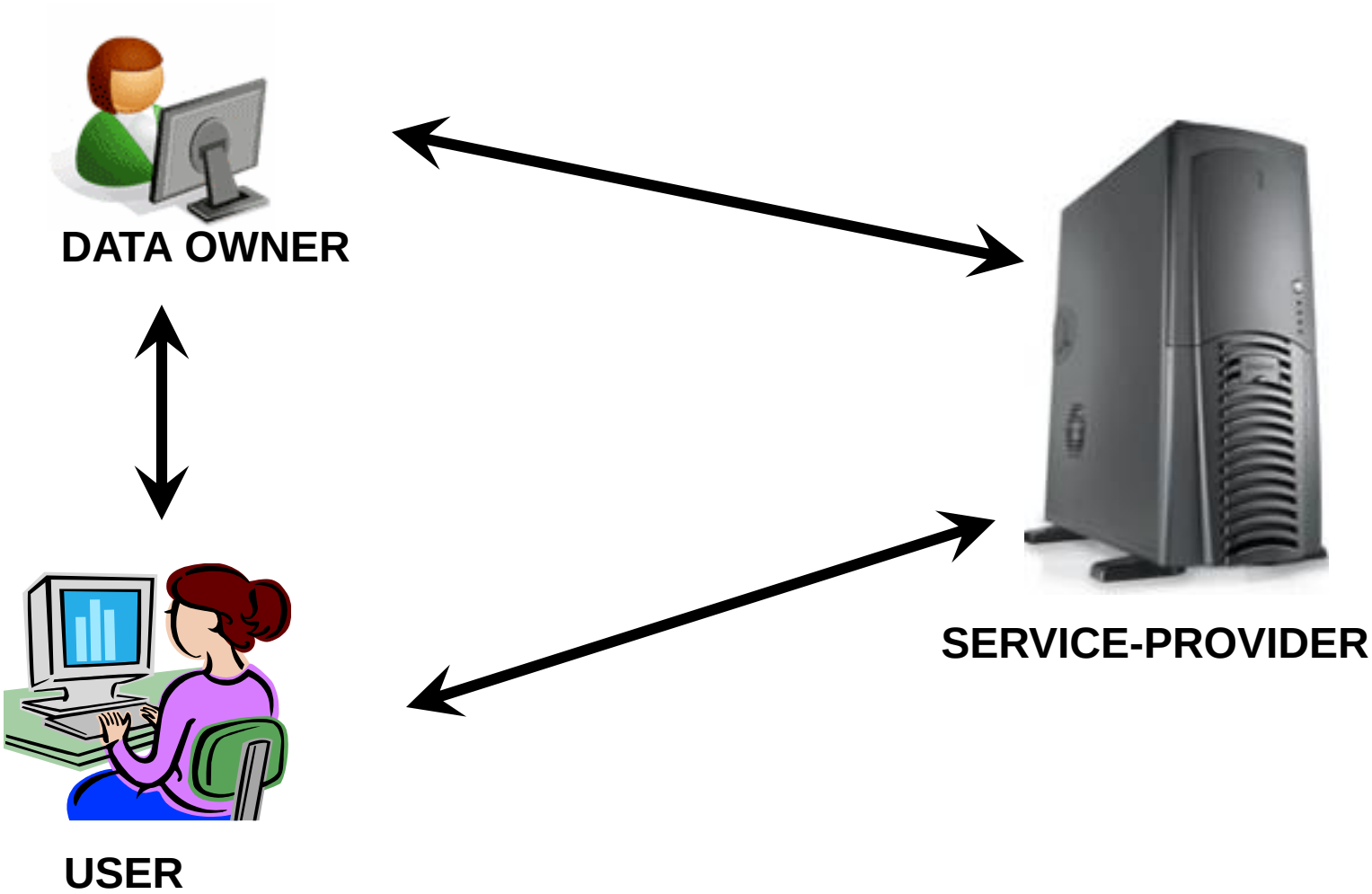
Towards Completeness

- Haber et al., "*Privacy-Preserving Computation and Verification of Aggregate Queries on Outsourced Databases*", in Privacy Enhancing Technologies Symposium, 2009.

- Sometimes it is undesirable to disclose specific data elements, due to privacy concerns.
- Instead, it may be acceptable to disclose aggregate statistics about the data.
  - E.g.: The statistic aggregates of confidential medical records of a group of patients might be accessible by the public; however, the medical record of individual patients should be kept private.

- Protocols for aggregate queries SUM, MAX, MIN, COUNT, AVERAGE & MEDIAN
- It allows a user to verify both correctness and completeness of aggregate results while the individual data values are kept secret.

Trust Model



Trust Model

- **Between data owner and service-provider**: must not disclose the data directly to a user, but only answer queries

- **Between service-provider and user**: user should be able to verify the response from service-provider

- **Between data owner and user**: user must trust any messages signed by the data owner

Functionality

SETUP PHASE

DATA OWNER

Sends database to service-provider

Plus auxiliary information

SERVICE-PROVIDER

QUERY PHASE

USER

Query

Response

SERVICE-PROVIDER

Query Completeness

- Completeness: The user (querier) can be sure that the query result is complete
  - No tuples that should be included in the result were left out
  - Tuples that should not be in the result are not included
- Achieved using several cryptographic building blocks:
  - Cryptographic commitments
  - Zero knowledge proofs of knowledge
  - Merkle hash trees

  (see article for more details)

Commitment Scheme

- Two entities involved: Sender and Receiver
- It is a method for the sender:
  - To choose a value
  - Temporarily hide it from the receiver
  - And then reveal it later to the receiver
  - (The initial value cannot be changed)
- Two phases:
  1. Commit: Sender selects a value, locks it in a box, and sends the locked box to the Receiver.
     - The box is the commitment
  2. Reveal: Sender proves to the Receiver that the box contains a certain value

Commitment Scheme

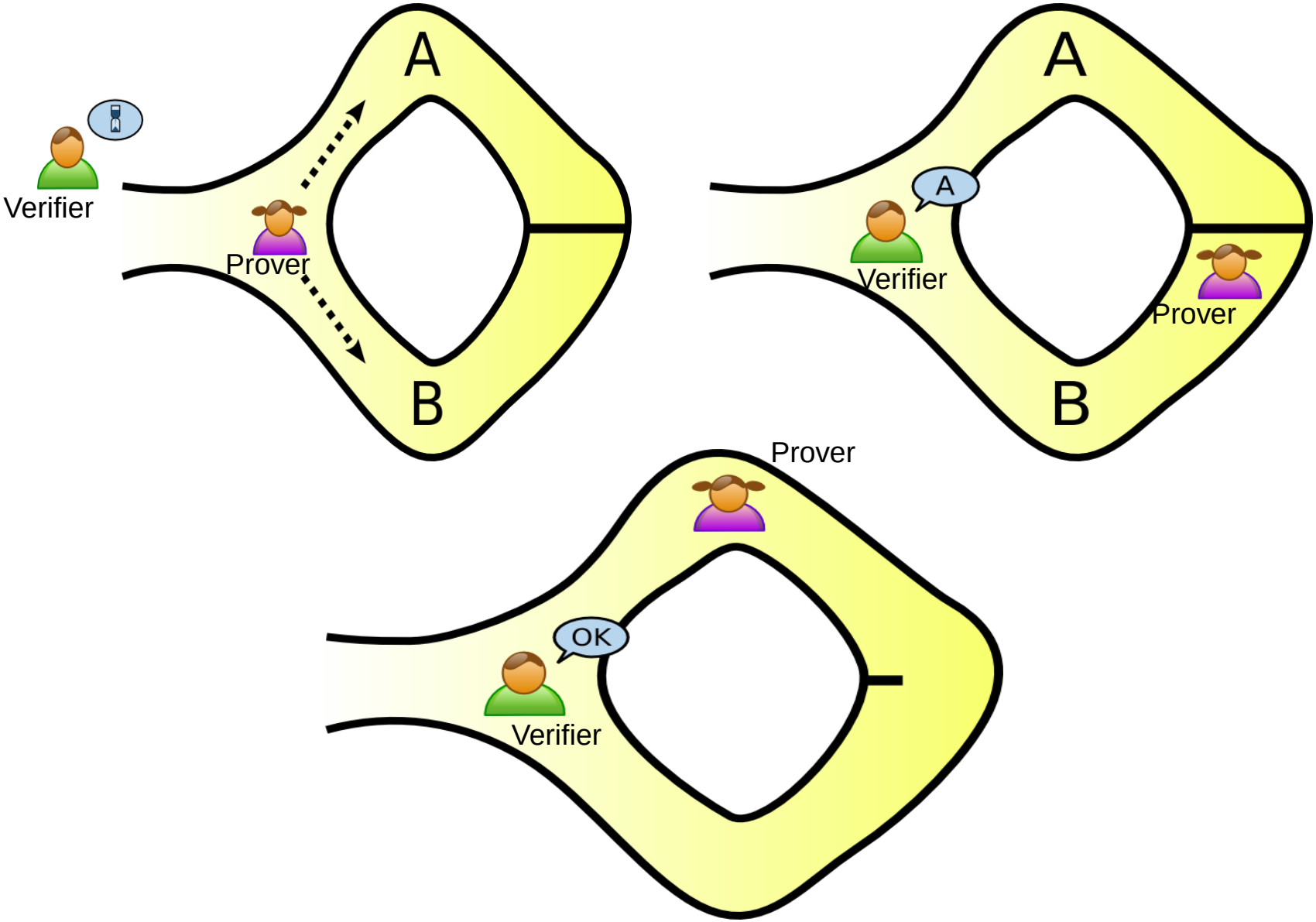
- A commitment scheme satisfies the following security properties:
  - Hiding property: at the end of the Commit phase, the Receiver learns no information about the committed value (the value locked in the box)
  - Binding property: at the end of the Reveal phase, the Sender cannot convince the Receiver that the box contains a different value than the one that was initially put in the box
    - In other words, the Sender cannot equivocate and lie about the value that was initially committed

Zero Knowledge Proofs

- Involves a Prover and a Verifier

- The Prover proves a statement to a Verifier without directly revealing the statement to the Verifier.

Zero Knowledge Proofs



ZOK in context with the paper

- The prover proves the aggregate result to a verifier without revealing the data value to the verifier.

- A zero-knowledge proof of knowledge allows a prover to demonstrate a verifier the knowledge of secret values or their relations

$$POK(x_1,r_1,x_2,r_2 | C_1=g^{x1}h^{r1}, C_2=g^{x2}h^{r2}, x_1-x_2>=0)$$

Summary

- DB outsourcing has cost-saving advantages, but introduces many security-related challenges
- When DB is outsourced at untrusted third parties, various security-related issues must be addressed
  - Confidentiality
  - Integrity
  - Completeness
  - Etc.

What to read?

- Database access control: Chapter 10.1.3 from the Textbook
- Protecting sensitive data: Chapter 10.1.4 from the Textbook
  - k-anonymity article: Latanya Sweeney, “k-ANONYMITY: A MODEL FOR PROTECTING PRIVACY”, available at https://epic.org/privacy/reidentification/Sweeney_Article.pdf
  - Other articles mentioned in this lecture